Portable check-in hardware: specs

Status as of **19 October 2026**. What the face, fingerprint and barcode ID card devices must meet to work with what is already built. The server side for all three is done; the contracts are in `card-check-in.md`, `fingerprint-check-in.md` and `face-recognition.md`. Values below come from those docs, `src/config/env.ts`, the attendance and verification schemas, and the frontend's `FaceCamera.tsx`.

---

1. What every terminal needs

Requirement	Why
Always online (Wi-Fi or 4G SIM)	There is no offline mode or store-and-forward. A terminal that cannot reach the API cannot record attendance
Programmable, speaks HTTPS	Card and fingerprint terminals `POST` JSON with an `X-API-Key` header (32+ characters). A plain USB reader cannot do this alone; it must sit behind a phone, Android handheld or microcontroller
A screen	The device must show the student's name on success, and messages such as "Already marked present" or "You are not registered for COSC 205"
A terminal id	Sent with each fingerprint check-in: letters, digits, `.`, `_`, `-`, up to 64 characters, starting with a letter or digit (e.g. `TERM-01`)
Safe retry in firmware	On a timeout, retry once and treat `409` as success; a retry cannot double-record. On `429`, wait and retry instead of dropping the scan
Fits the per-IP limit	All terminals behind one campus NAT share `CHECKIN_RATE_LIMIT_PER_IP` (default 3,000 per 5 minutes). Fine for normal classes; a reader stuck in a loop will hit it
A way to receive the session id	Every check-in names the class session, handed over by the lecturer's screen. **Not built yet:** the id only appears in the lecturer's page URL (`ActivateClass.tsx`), with no QR or code for a terminal to read

---

2. Face recognition

Spec	Requirement
Device	Currently the lecturer's own phone, signed in, running `/session/:id/face` in the browser
Camera	Rear camera; 1280×720 is requested, photos are scaled to 720 px
Face size	At least 80 px in the scaled photo (`FACE_MIN_FACE_PX`), roughly arm's length
Image quality	Sharpness 40 or more (`FACE_MIN_SHARPNESS`); JPEG, PNG or WebP, about 300 KB maximum per upload
Liveness	Not needed in v1: the lecturer confirms each match in person
Processing	On the server (OpenCV YuNet + SFace in `face-service/`). The device only captures

Conclusion: any mid-range Android phone or tablet with a decent rear camera and Chrome. No special face hardware is needed.

Known risk: `ATTENDANCE_fronted/deploy/default.conf.template` sets `Permissions-Policy: camera=(), microphone=(), geolocation=()`, which blocks the camera (and geofence) if the site is served through that nginx config.

---

3. Fingerprint

Designed so that **no fingerprint data reaches the server.** The reader stores templates, runs the 1:N match itself, and sends only a slot reference:

```
{ "sessionId": "...", "terminalId": "TERM-01", "fingerRef": "37" }
```

Spec	Requirement
Matching	On the device, 1:N, against templates stored on the device
Output	A slot or reference id: letters, digits, `:`, `_`, `-`, up to 64 characters
Readers the docs target	R307, ZFM-20, GROW R503 or similar
Template capacity	The key spec for a portable unit. A slot only means something on the reader that enrolled it, so a terminal carried between classes must hold every student who may use it. R503 holds about 200, R307 about 1,000. For a whole faculty, use an Android handheld whose app stores templates locally (this still fits the contract)
Sensor	Optical or capacitive, tolerant of dry or dusty fingers; FAP20+ class preferred for large groups

Operational cost: each student must be enrolled on each device *and* mapped on the server. Server mapping is script-only today (`npm run dev:enrol-fingerprint`). Revoking on the server does not clear the template off the reader.

---

4. ID card scanning (decided: barcode)

The university's student ID cards carry a **barcode**, not an NFC chip. That changes both the hardware and the backend.

Spec	Requirement
Scanner	1D + 2D imager (reads Code 128, Code 39 and QR). A 2D imager also lets the terminal scan a QR from the lecturer's screen, which solves the session id handoff (see 6.1)
Read from	Printed plastic cards, and phone screens (for the session QR)
Output	The decoded text, sent as-is in the check-in request

Backend does not accept barcodes yet. `cardCheckInSchema` only allows hex (`/^[0-9a-fA-F][0-9a-fA-F\s:-]*$/`), and `normaliseCardUid` strips `-`. A registration-number barcode such as `SCT211-0001/2022` gets a `400`. Needed:

- A barcode check-in route (or a `kind` field) that accepts the card's text format, matched case-insensitively, without stripping `-` or `/`.
- **Confirmed: the barcode is the student's registration number.** Students are matched on `student_profiles.registration_number` (stored upper-case). No per-card enrolment is needed, so `student_cards` stays for NFC only.

Decided: any one method per student. Barcode, fingerprint, face and QR all run at once in a session so large classes clear quickly; each student uses whichever one they like, and the one-record-per-session rule stops a second check-in. Accepted risk: a photo or photocopy of a classmate's card is enough to check them in with the barcode, so lecturers should keep an eye on the terminal.

---

5. Hardware options (exploring both)

Each terminal must do **barcode + fingerprint**, with Wi-Fi as the main link and 4G as backup. Face check-in stays on the lecturer's phone either way.

Terminals are wireless and portable, not fixed. Each one belongs to a department's pool. A lecturer collects one, carries it to class, and links it to the session by scanning a QR code on the lecturer's screen. A terminal may only be linked to classes taught by its own department's lecturers.

Option A: Android biometric handheld

Part	Spec
Device	Android 11+ rugged handheld with built-in fingerprint sensor (FAP20+, SDK with on-device 1:N matching) and 1D/2D barcode imager
Network	Wi-Fi (2.4 + 5 GHz) and 4G LTE with SIM slot; Android falls back automatically
Battery	5,000 mAh or more, enough for a teaching day
Screen	5.5 to 8 inches
Build	IP65 or better, survives a 1.2 m drop
Templates	Stored in the device app, thousands per device
Software	Small terminal app (**to be built**); face page runs in Chrome

Indicative price: **USD 280 to 340 per unit** wholesale from Alibaba (USD 500 for a single sample). Landed in Kenya, after shipping, duty and 16% VAT, estimate roughly **KSh 50,000 to 70,000 per unit**. No Kenyan retailer listed an exact match, so this needs quotes. The nearest local item is a 4G Android POS handheld at KSh 45,000 + VAT (Mtech), with no fingerprint sensor.

Option B: DIY box

Part	Spec	Indicative price
Controller	ESP32 (Wi-Fi) or Raspberry Pi Zero 2 W	estimate, needs local quote
Fingerprint	GROW R503, capacitive, 200 templates	USD 16 (AliExpress) to USD 40 (Adafruit)
Barcode	GM65 or similar 1D/2D module (also reads QR)	about SGD 78 / INR 3,100 retail abroad

Part	Spec	Indicative price
4G backup	SIM7600 LTE module	estimate, needs quote
Other	Screen, battery pack, case	estimate

Estimated **KSh 12,000 to 20,000 per box** in parts, plus build time, firmware, and someone to repair the boxes. All prices are from other markets and leave out shipping and duty.

How the options fit your setup

Factor	Option A	Option B
Fingerprint capacity	Thousands of students	**200 students.** This rules Option B out for a department pool, which must hold every student taking the department's units. Each box would only cover one large class
Wi-Fi + 4G fallback	Built in	Extra module and firmware work
Session QR handoff	Built-in imager	GM65 reads QR
Robustness / support	Commercial, under warranty	Home-built, kept running in-house
Cost per unit	Higher (estimate KSh 50k to 70k)	Lower (estimate KSh 12k to 20k)
Software to build	Android terminal app	ESP32/Pi firmware

Recommendation: use Option A for production. Build one Option B box as a cheap prototype to test the backend contract before the handhelds arrive.

Throughput

At about 3 seconds per student, one terminal handles a class of 200 in roughly 10 minutes. Plan two terminals for classes over about 100. Doubling the check (barcode then fingerprint) roughly doubles this.

---

6. Build plan

Order: 6.1 first because it is small and stands alone; 6.2 before 6.3 because fingerprint enrolment is tied to a registered terminal.

Frontend (built): the lecturer's "Link a terminal" button on the live session screen (6.2) and the department portal's Terminals and Fingerprint enrolment pages (6.3).

Terminal software (built, alongside the backend and frontend repos): `ATTENDANCE_terminal_android` (Option A; fingerprint reader is a pluggable interface with a simulated stand-in until the handheld model is chosen) and `ATTENDANCE_terminal_esp32` (Option B: R503 + GM65 + OLED, Wi-Fi only).

6.1 Barcode check-in (built: `POST /attendance/barcode-check-in`, see `card-check-in.md` §4)

- New terminal route that accepts the registration number as printed (letters, digits, `/`, `-`), trimmed and upper-cased, never stripped.
- Recorded as the existing `CARD` method, so the lecturer's "ID card" toggle stays as it is.
- A student with no attendance account yet has it created from SMARTTT on their first scan (decided 19 Oct 2026), so nobody has to sign in first.

6.2 Terminal registration and setup (built: see `terminals.md`)

- `terminals` table: code (e.g. `TERM-01`), owning department, capabilities (barcode, fingerprint), hashed per-device key, status (active, lost, retired), last seen, battery, network, linked session.
- **Pairing:** a department officer adds the device in the portal and gets a one-time QR code; the device scans it and receives its own key.
- **Linking to a class:** the lecturer's screen shows a short-lived QR code; the terminal scans it. The server checks the class is taught by a lecturer in the terminal's department. The link ends when the class closes.
- **Reporting in:** the device regularly sends battery and network (Wi-Fi or 4G) and receives its linked class, allowed methods and pending enrolments.
- Rate limits per device rather than per IP address.
- The shared `CARD_TERMINAL_API_KEY` and `FINGERPRINT_TERMINAL_API_KEY` keep working until every device is paired, then are removed.

6.3 Department screens (built: backend in `terminals.md`, portal pages Terminals and Fingerprint enrolment)

- **Devices:** register, pair, rename, mark lost (key stops working at once), retire; see last seen, battery and linked class.
- **Fingerprint enrolment:** the officer picks a student and a device; the device picks up the pending enrolment when it reports in, takes the finger, and sends back the slot it used. No fingerprint data reaches the server and no officer logs in on the device.
- **Revoking** queues a "delete this slot" instruction for the device.
- **Which students:** those with active allocations on units taught by the department's lecturers, the same rule the department's `/students` page uses.
- **Coverage view:** which students are enrolled on which devices.

6.4 Later

- SMARTTT already stores a department on each unit and each student, but its integration endpoints do not send either. Adding the unit's department to `attendance/lecturer-units/` would let attendance scope by the unit's own department, for when a lecturer teaches another department's unit.
- Dedicated device credentials for face check-in instead of the lecturer's login (listed under "Later" in `face-recognition.md`).

---

7. Decisions

Question	Decision
Student ID card technology	**Barcode holding the registration number** (19 Oct 2026)
Methods per session	**Barcode, fingerprint, face and QR all on at once; each student uses any one**
Hardware option	**Exploring both A and B**; recommendation above
Terminal type	**Wireless and portable**, pooled per department
Which classes a terminal may serve	**Only classes taught by its department's lecturers**
Who enrols fingerprints	**The department**, for students on its lecturers' units
Connectivity	**Campus Wi-Fi first, 4G SIM as backup**